

Wpnote

目录

- [Wpnote](#)
 - [目录](#)
 - [1. 数据结构与基础算法](#)
 - [1.1 快速输入输出](#)
 - [1.2 调试宏](#)
 - [1.3 IsPrime 判断素数](#)
 - [2. 动态规划 \(DP\)](#)
 - [2.1 01 背包 \(一维/二维\)](#)
 - [一维数组版](#)
 - [二维数组版](#)
 - [2.2 完全背包](#)
 - [2.3 最大子段和](#)
 - [2.4 数字三角形](#)
 - [2.5 记忆化搜索](#)
 - [3. 图论与搜索](#)
 - [3.1 DFS 深度优先搜索](#)
 - [小猫爬山](#)
 - [3.2 无向图分组](#)
 - [4. 二分与双指针](#)
 - [4.1 二分搜索](#)
 - [货运难题](#)
 - [4.2 双指针](#)
 - [Cli 与字符串](#)
 - [5. 差分与前缀和](#)
 - [5.1 差分 + 前缀和](#)
 - [5.2 重叠线段覆盖](#)
 - [6. 贪心算法](#)
 - [6.1 哈夫曼编码](#)
 - [7. Trick 与数据抽象](#)
 - [7.1 Map 前缀和](#)
 - [7.2 数据抽象化构造](#)
 - [8. 常见错误与坑点](#)

1. 数据结构与基础算法

1.1 快速输入输出

用途：加速 C++ 输入输出，防止超时。

常见坑：关闭同步后，勿混用 cin/cout 和 scanf/printf。

```
ios::sync_with_stdio(false);
cin.tie(nullptr);
cout.tie(nullptr);
```

复杂度： $O(1)$ 初始化。

1.2 调试宏

用途：本地调试，打印变量值。

常见坑：提交 OJ 时注释掉 `#define LOCAL_DEBUG`。

```
#ifdef LOCAL_DEBUG
#define dbg(...) cerr << "DEBUG: [" << #__VA_ARGS__ << "] = [" ,
    _print(__VA_ARGS__)
void _print() { cerr << "]\n"; }
template <typename T, typename... V>
void _print(T t, V... v) {
    cerr << t;
    if (sizeof...(v)) cerr << ", ";
    _print(v...);
}
#else
#define dbg(...) ;
#endif
```

复杂度： $O(1)$ 每次调用。

1.3 IsPrime 判断素数

用途：判断一个数是否为素数。

常见坑：注意边界 ($a < 2$ 或 $a = 2$)，避免大数溢出。

```
bool isPrime(int a) {
    if (a == 2 || a == 3) return true;
    if (a < 2 || a % 2 == 0) return false;
    for (int i = 3; i * i <= a; i += 2)
        if (a % i == 0) return false;
    return true;
}
```

复杂度： $O(\sqrt{n})$ 。

2. 动态规划 (DP)

2.1 01 背包（一维/二维）

用途：每个物品选或不选，求最大价值（不要求装满/要求恰好装满）。

常见坑：

- 一维数组需倒序循环，防止覆盖。
- 恰好装满需初始化 $dp[0]=0$, $dp[1\dots v]=-INF$ 。
- 注意 long long 防溢出。

一维数组版

```
void solve() {
    int n, v;
    cin >> n >> v;
    struct kind { int value, weight; };
    vector<kind> nums(n);
    for (int i = 0; i < n; i++) {
        cin >> nums[i].weight >> nums[i].value;
    }
    vector<int> dp(v + 1);
    vector<int> dp2(v + 1, -99999999); // 恰好装满
    dp2[0] = 0;
    for (int i = 0; i < n; i++) {
        for (int j = v; j >= nums[i].weight; j--) {
            dp[j] = max(dp[j], dp[j - nums[i].weight] + nums[i].value);
            dp2[j] = max(dp2[j], dp2[j - nums[i].weight] + nums[i].value);
        }
    }
    cout << dp[v] << '\n';
    cout << (dp2[v] > 0 ? dp2[v] : 0) << '\n';
}
```

二维数组版

```
void solve() {
    int t, m; // t=容量, m=物品数
    cin >> t >> m;
    struct cy { int time, ac; };
    vector<cy> cord(m + 1);
    for (int i = 1; i <= m; i++) {
        cin >> cord[i].time >> cord[i].ac;
    }
    int dp[105][1005] = {};
    for (int i = 1; i <= m; i++) {
        for (int j = 0; j <= t; j++) {
            if (j >= cord[i].time)
                dp[i][j] = max(dp[i-1][j-cord[i].time] + cord[i].ac, dp[i-1][j]);
            else
                dp[i][j] = dp[i-1][j];
        }
    }
}
```

```

    }
}
cout << dp[m][t] << '\n';
}

```

复杂度:

- 一维: $O(n*v)$, 空间 $O(v)$ 。
- 二维: $O(nt)$, 空间 $O(nt)$ 。

2.2 完全背包

用途: 物品可无限选, 求最大价值。

常见坑: 正序循环 (区别于 01 背包倒序), 防止重复选择。

```

void solve() {
    int time, n;
    cin >> time >> n;
    struct cy { int t, v; };
    vector<cy> hsh(n + 1);
    for (int i = 1; i <= n; i++)
        cin >> hsh[i].t >> hsh[i].v;
    vector<int> dp(time + 1);
    for (int i = 1; i <= n; i++) {
        for (int j = hsh[i].t; j <= time; j++)
            dp[j] = max(dp[j - hsh[i].t] + hsh[i].v, dp[j]);
    }
    cout << dp[time] << '\n';
}

```

复杂度: $O(n*time)$, 空间 $O(time)$ 。

2.3 最大子段和

用途: 求数组连续子段的最大和。

常见坑: 负数子段需重置, 注意初始化。

```

void solve() {
    int n;
    cin >> n;
    vector<int> nums(n + 1);
    for (int i = 1; i <= n; i++) cin >> nums[i];
    vector<int> dp(n + 1);
    int ans = -4545556565;
    for (int j = 1; j <= n; j++) {
        dp[j] = dp[j-1] + nums[j];
        if (dp[j] <= 0) dp[j] = nums[j];
    }
}

```

```

        ans = max(ans, dp[j]);
    }
    cout << ans << '\n';
}

```

复杂度： $O(n)$ ，空间 $O(n)$ 。

2.4 数字三角形

用途：从顶到底或底到顶，求最大路径和。

常见坑：从底到顶递推可省去边界处理。

```

void solve() {
    int n;
    cin >> n;
    int sjx[100][100];
    for (int i = 0; i < n; i++)
        for (int j = 0; j <= i; j++)
            cin >> sjx[i][j];
    for (int i = n-2; i >= 0; i--)
        for (int j = 0; j <= i; j++)
            sjx[i][j] += max(sjx[i+1][j], sjx[i+1][j+1]);
    cout << sjx[0][0] << '\n';
}

```

复杂度： $O(n^2)$ ，空间 $O(1)$ （直接在输入数组上修改）。

2.5 记忆化搜索

用途：从未知到已知递推，适合复杂状态转移。

与 DP 区别：

- DP：从已知到未知，严格顺序，循环实现，常数小。
- 记忆化搜索：从未知到已知，递归实现，需状态数组防重复计算。

```

vector<int> mama(100230, -1);
int dfs(int a) {
    if (a == 1) return 1;
    if (mama[a] != -1) return mama[a];
    int cnt = 1;
    for (int i = 1; i <= a / 2; i++)
        cnt += dfs(i);
    mama[a] = cnt;
    return cnt;
}
void solve() {
    int n;

```

```

    cin >> n;
    cout << dfs(n) << '\n';
}

```

复杂度： $O(n^2)$ ，空间 $O(n)$ （状态数）。

3. 图论与搜索

3.1 DFS 深度优先搜索

用途：枚举、路径搜索、连通性。

常见坑：注意回溯还原（如标记数组清零），加剪枝防超时。

小猫爬山

```

int n, w, ans = 20;
vector<int> lcs(20);
void dfs(int stepnum, int lcnum, const vector<int> &cats) {
    if (lcnum >= ans) return;
    if (stepnum == n) {
        ans = lcnum;
        return;
    }
    for (int j = 1; j <= lcnum; j++)
        if (lcs[j] + cats[stepnum] <= w) {
            lcs[j] += cats[stepnum];
            dfs(stepnum + 1, lcnum, cats);
            lcs[j] -= cats[stepnum];
        }
    lcs[lcnum + 1] = cats[stepnum];
    dfs(stepnum + 1, lcnum + 1, cats);
    lcs[lcnum + 1] = 0;
}
void solve() {
    cin >> n >> w;
    vector<int> cats(n + 1);
    for (int i = 0; i < n; i++) cin >> cats[i];
    sort(cats.rbegin(), cats.rend());
    dfs(0, 1, cats);
    cout << ans << '\n';
}

```

复杂度： $O(2^n)$ 最坏，剪枝后更优。

3.2 无向图分组

用途：求连通分量数。

常见坑：双向边需存两次，初始化标记数组。

```

vector<vector<int>>> a(2e5 + 10);
vector<int> kk(2e5 + 10);
void dfs(int k) {
    for (int i = 0; i < a[k].size(); i++)
        if (kk[a[k][i]] == 0) {
            kk[a[k][i]] = 1;
            dfs(a[k][i]);
        }
}
void solve() {
    int n, m, ans = 0;
    cin >> n >> m;
    for (int i = 0; i < m; i++) {
        int p, q;
        cin >> p >> q;
        a[p].push_back(q);
        a[q].push_back(p);
    }
    for (int j = 1; j <= n; j++)
        if (kk[j] == 0) {
            kk[j] = 1;
            dfs(j);
            ans++;
        }
    cout << ans << '\n';
}

```

复杂度： $O(n+m)$, 空间 $O(n+m)$ 。

4. 二分与双指针

4.1 二分搜索

用途： 答案单调时逼近最优解。

常见坑：

- 最小值模板：if(check) r=mid else l=mid+1。
- 最大值模板：if(check) l=mid else r=mid-1。
- 注意边界（left 初始化为最大单元素）。

货运难题

```

int n, k;
bool check(int pans, const vector<int> &nums) {
    int hsh = 0, cnt = 0;
    for (int i = 0; i < n; i++) {
        if (hsh + nums[i] > pans) {
            cnt++;
        }
    }
}

```

```

        hsh = nums[i];
    } else
        hsh += nums[i];
    }
    return cnt <= k-1;
}

void solve() {
    cin >> n >> k;
    vector<int> nums(n);
    int sum = 0, zd = 0;
    for (int i = 0; i < n; i++) {
        cin >> nums[i];
        sum += nums[i];
        zd = max(zd, nums[i]);
    }
    int left = zd, right = sum;
    while (left < right) {
        int mid = (left + right) / 2;
        if (check(mid, nums)) right = mid;
        else left = mid + 1;
    }
    cout << left << '\n';
}

```

复杂度： $O(n \cdot \log(\text{sum}))$ ，空间 $O(n)$ 。

4.2 双指针

用途：滑动窗口、区间统计。

常见坑：防止越界，注意快慢指针逻辑。

Cli 与字符串

```

void solve() {
    int n, k;
    cin >> n >> k;
    string s;
    cin >> s;
    int ans = 1e8;
    if (k == 1) {
        cout << "1";
        return;
    }
    vector<int> jjj;
    for (int i = 0; i < 26; i++) {
        int cnt = 0, l = 0;
        for (int r = 0; r < s.size(); r++) {
            if ((int)(s[r] - 'a') == i) cnt++;
            while (cnt >= k && l <= r) {
                if ((int)(s[l] - 'a') == i) {

```



```

        ans = min(r - l + 1, ans);
        cnt--;
    }
    l++;
}

}

}
    jjj.push_back(ans);
}

sort(jjj.begin(), jjj.end());
cout << (jjj[0] == 1e8 ? -1 : jjj[0]) << '\n';
}

```

复杂度： $O(n \cdot 26)$ ，空间 $O(26)$ 。

5. 差分与前缀和

5.1 差分 + 前綴和

用途：区间修改后求点值或最小值。

常见坑：差分数组需初始化，注意边界。

```

struct niubi {
    int l, r, gap;
};

void solve() {
    int n, p;
    cin >> n >> p;
    vector<int> c(n + 1);
    for (int i = 1; i <= n; i++) cin >> c[i];
    vector<int> hsh(5e6 + 1);
    for (int i = 1; i <= n; i++) hsh[i] = c[i] - c[i-1];
    vector<niubi> nums(p + 1);
    for (int i = 1; i <= p; i++)
        cin >> nums[i].l >> nums[i].r >> nums[i].gap;
    for (int i = 1; i <= p; i++) {
        hsh[nums[i].l] += nums[i].gap;
        hsh[nums[i].r + 1] -= nums[i].gap;
    }
    vector<int> hsh2(5e6 + 1);
    hsh2[1] = hsh[1];
    for (int i = 2; i <= n; i++)
        hsh2[i] = hsh2[i-1] + hsh[i];
    int ans = 201;
    for (int i = 1; i <= n; i++)
        ans = min(ans, hsh2[i]);
    cout << ans << '\n';
}

```

复杂度: $O(n+p)$, 空间 $O(n)$ 。

5.2 重叠线段覆盖

用途：计算多段重叠后的总覆盖长度。

常见坑：排序后处理重叠，注意边界。

```
void solve() {
    int n;
    cin >> n;
    vector<int> l(n), r(n);
    for (int i = 0; i < n; i++)
        cin >> l[i] >> r[i];
    sort(l.begin(), l.end());
    sort(r.begin(), r.end());
    int ans = 0;
    for (int i = 0; i < n; i++) {
        ans += r[i] - l[i];
        if (i + 1 < n && r[i] >= l[i + 1])
            ans -= (r[i] - l[i + 1]);
    }
    cout << ans << '\n';
}
```

复杂度： $O(n \log n)$ ，空间 $O(n)$ 。

6. 贪心算法

6.1 哈夫曼编码

用途：用优先队列合并最小代价。

常见坑：优先队列默认大顶堆，需用 `priority_queue<int, vector<int>, greater<int>>`。

```
int main() {
    ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    int n, sum = 0;
    priority_queue<int, vector<int>, greater<int>> q;
    cin >> n;
    while (n--) {
        int i;
        cin >> i;
        q.push(i);
    }
    while (q.size() > 1) {
        int s = q.top(); q.pop();
        s += q.top(); q.pop();
        q.push(s);
        sum += s;
    }
    cout << sum << endl;
}
```

```
    return 0;
}
```

复杂度: $O(n \log n)$, 空间 $O(n)$ 。

7. Trick 与数据抽象

7.1 Map 前缀和

用途: 用 Map 记录前缀和, 求最短满足条件的子串。

常见坑: 初始化 Map, 处理空字符串情况。

```
void solve() {
    int n, sum = 0;
    cin >> n;
    string s;
    cin >> s;
    map<int, int> caonima;
    caonima[0] = -1;
    int niubi = 0, ans = n;
    for (int i = 0; i < n; i++) {
        niubi += s[i] == 'a' ? 1 : -1;
        caonima[niubi] = i;
        if (caonima.count(niubi - sum))
            ans = min(ans, i - caonima[niubi - sum]);
    }
    cout << (ans == n ? -1 : ans) << '\n';
}
```

复杂度: $O(n \log n)$, 空间 $O(n)$ 。

7.2 数据抽象化构造

用途: 根据贡献构造数组, 处理复杂关系。

常见坑: 注意边界 ($i=1$ 时特殊处理)。

```
void solve() {
    int n;
    cin >> n;
    vector<int> b(n + 1, 0);
    for (int i = 1; i <= n; i++) cin >> b[i];
    vector<int> aa(n + 1);
    for (int i = 1; i <= n; i++) {
        int kk = (i == 1 ? 0 : b[i-1]);
        int pp = b[i] - kk;
        int pos = i - pp;
        aa[i] = (pos <= 0 ? i : aa[pos]);
    }
}
```

```
        cout << aa[i] << ' ';  
    }  
    cout << '\n';  
}
```

复杂度： $O(n)$ ，空间 $O(n)$ 。

8. 常见错误与坑点

- **二分搜索：**最小值模板用 $r=mid/l=mid+1$ ，最大值用 $l=mid/r=mid-1$ 。注意 check 函数逻辑。
 - **DFS：**回溯需还原（如标记数组清零），加剪枝（如 $lcnum \geq ans$ ）。
 - **边界处理：**while 循环防越界（如 $r < n$ ），初始化防空集合。
 - **溢出：**用 `long long`（取消 `#define int long long` 后检查）。
 - **DP 初始化：**恰好装满背包需 $dp[0]=0$ ，其他负无穷。
 - **快慢指针：**确保 $l \leq r$ ，防止越界。
-